

Empirical Seminar - Biased ViT Patches

Itamar Sabban

June 24, 2022

1 Introduction

Attention and Transformer based architectures such as BERT have been a staple in the field of NLP for a while now. Recently such architectures were found to be effective in a number of other fields of deep learning. As described in "An image is worth 16x16 words" from 2020, BERT-like architectures have proven to be effective in the field of image processing and classification when operating on embedded image patches. These architectures were found to be better than big ResNets which were the SOTA architectures for image classification at the time when trained on large enough datasets. The hypothesized reason was that Transformers lack image-specific inductive biases that are present in convolution based networks such as locality, two-dimensional neighborhood structure and translation equivariance. While this makes Transformer based architectures "slower learners" than CNNs, when trained on a sufficient amount of data they are able to reach better results since their lack of bias allows them to learn better, more general functions that might contradict these biases.

My hypothesis in this experiment is that changing the division of images into patches in certain ways (using bigger patches near the middle of the image for example), might inject image-specific bias into the model and therefore achieve better results in the earlier phases of training. As shown in this experiment, this might be somewhat true as when trained over CIFAR-10 most of the biased models achieve better accuracies than the unbiased model in the first few training epochs, but the unbiased models end up with the best accuracies after 25 training epochs. I will try to explain this phenomenon in this paper.

This might have some significance as it can serve as a very simple and relatively cheap way to inject bias into Transformer-based image models and achieve better results when trained over a small dataset, or with a low number of epochs.

2 Prior Works

As described in the introduction chapter, the main work this experiment is inspired by and relates to is "An image is worth 16x16 words" and their Vision Transformer architecture from 2020. Their model was proposed as an alternative to CNNs that were the SOTA image models at the time, and when trained on large enough datasets (such as their enormous JFT-300M dataset with 300 million samples of tagged images) and finetuned on smaller datasets, achieved higher accuracies than said CNNs. Their main insight was that this was the result of the lack of image-specific bias in Transformer models. This result prompted me to experiment with using uneven batch divisions of images as a way to inject bias into the model.

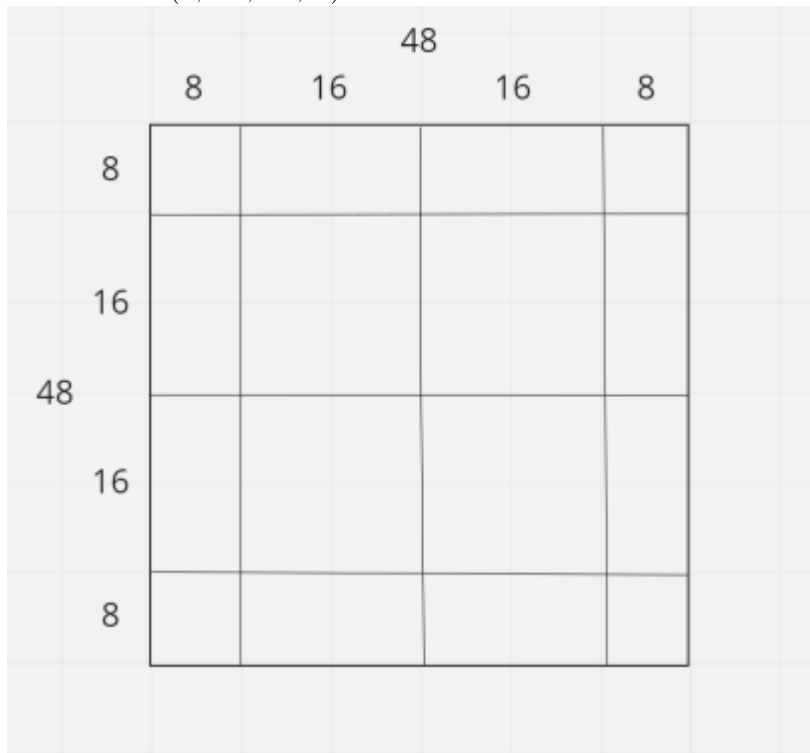
ViT was inspired by the BERT architecture from 2018's "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" which was based on the Transformer architecture presented in 2017's "Attention is all you need".

3 Method

All of the experiments were performed on the CIFAR-10 dataset (resized to 48x48 for convenience when dividing to patches). Most of the model architecture stays identical to the original Vision Transformer. Position embeddings are learned and added to the embedded patch tokens, and after concatenating the learned class token (with its own learned position embedding) to the token sequence, it is passed through 3 encoder transformer layers, each with 8 heads, and then the class token output is passed through a linear layer and the output classification logits are calculated. The architecture differs from the original Vision Transformer in its ability to allow for biased, uneven patch embeddings. In practice this is done using one convolutional layer with kernel size and stride equal to the highest patch size which is applied to the image after adding rows and columns of zeros to it so that the convolution kernel lands on each of the patches (possibly padded with some zero rows and columns). Note that the current shape of the output is (batch_size, d, sqrt(N), sqrt(N)) where d is the embedding dimension and N is the number of patches (which should be a perfect square). The last two dimensions are then flattened, and then the new last two dimensions are swapped, so the final patch embedding output is of shape (batch_size, N, d). Note that using this method, the patches are embedded using the same kernel (and thus using the same parameters) despite being of different shapes which necessitates the use of padding. In order to check if this had a negative effect on the learning, I created an alternative embedding method that embeds each patch with a separate linear layer, each with

an input size equal to the number of pixels in the patch, so no padding is necessary. This alternative method will be referred to as “seperate patch embeddings” in the rest of this paper.

Patch sizes (8, 16, 16, 8) visualized:



4 Experiments

My first experiments were short training and testing runs (2-3 epochs) with varying patch sizes and hyper parameters, with the main intent of verifying or disproving my hypothesis, and gaining some hyper-parameters intuition.

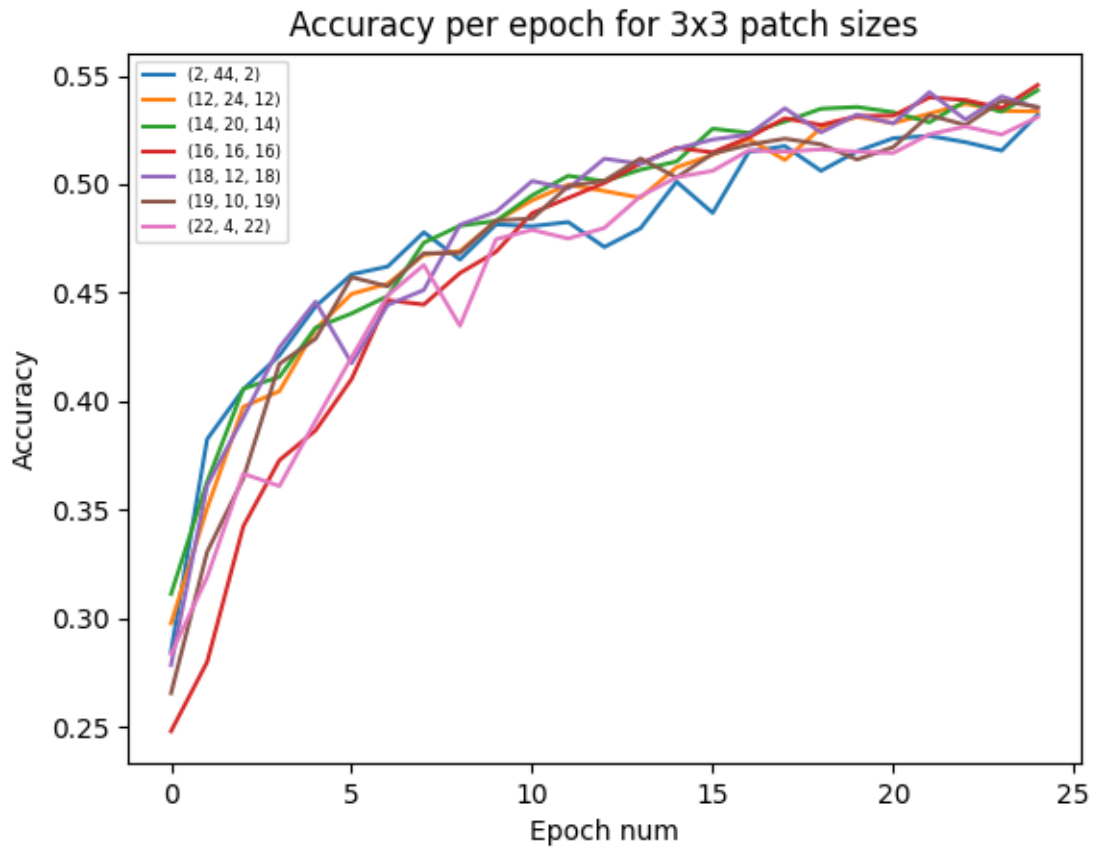
Relating to hyper-parameters, the main insights were that using weight decay and dropout consistently reduced accuracy and that SGD was vastly superior to Adam here (these remained true for longer training and testing runs of 25 epochs). Other than that the chosen hyper-parameters were learning rate = 0.001, batch size = 100 and 3 encoder layers each with 8 heads. For the main experiment I chose an embedding dimension d of 256 (the encoder feed-forward dimension for all exper-

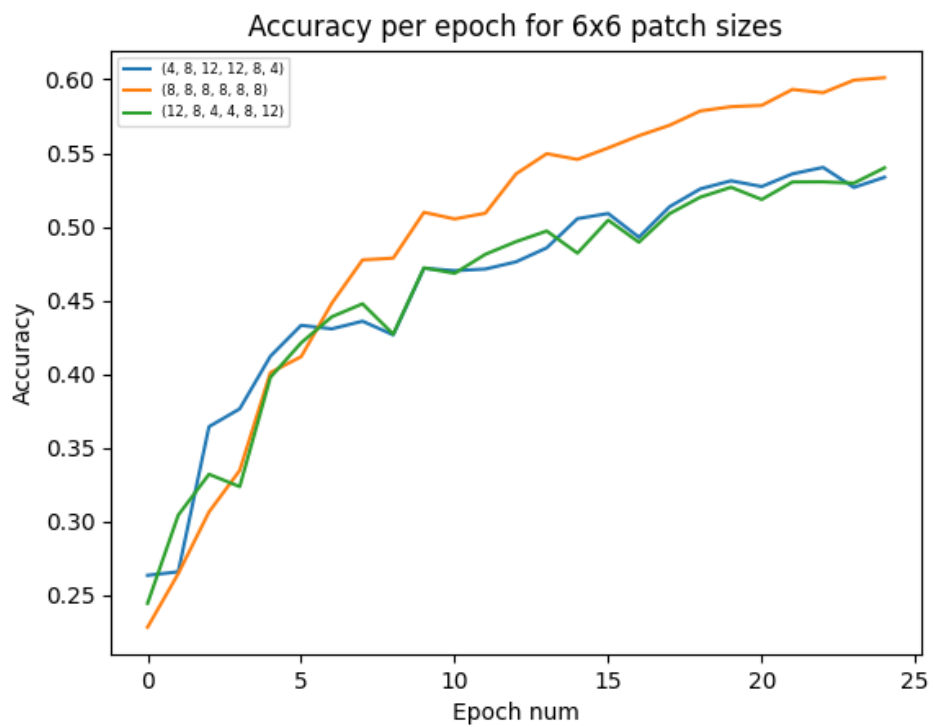
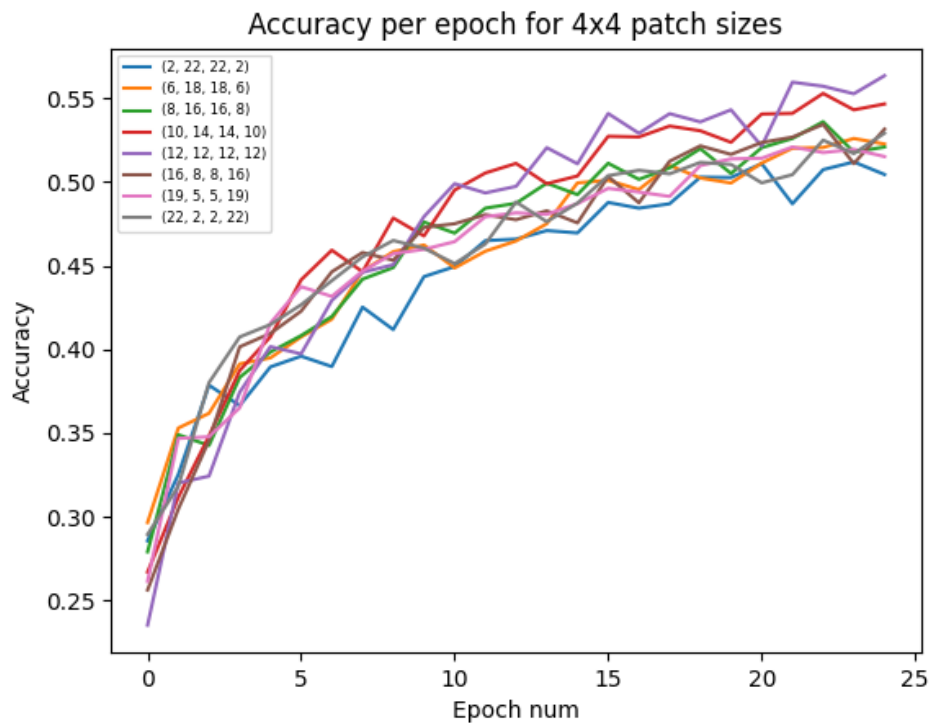
iments was $2*d$), as it was the highest d value with which the experiments took a reasonable amount of time.

As for verifying or disproving the hypothesis, the first toy experiments showed great promise, as the biased models beat the classic model nearly 100% of the time.

The main experiment I performed involved training 17 non-separate patch embeddings models, each with different patch sizes, for 25 epochs, and comparing their test accuracy after each epoch. Additionally, the cosine similarity of the learned position embeddings in each of the models after the training phase ended was calculated. 3 of the models included unbiased, even patches, and the rest were biased.

The accuracy results:





Some of the position embeddings' cosine similarity results:

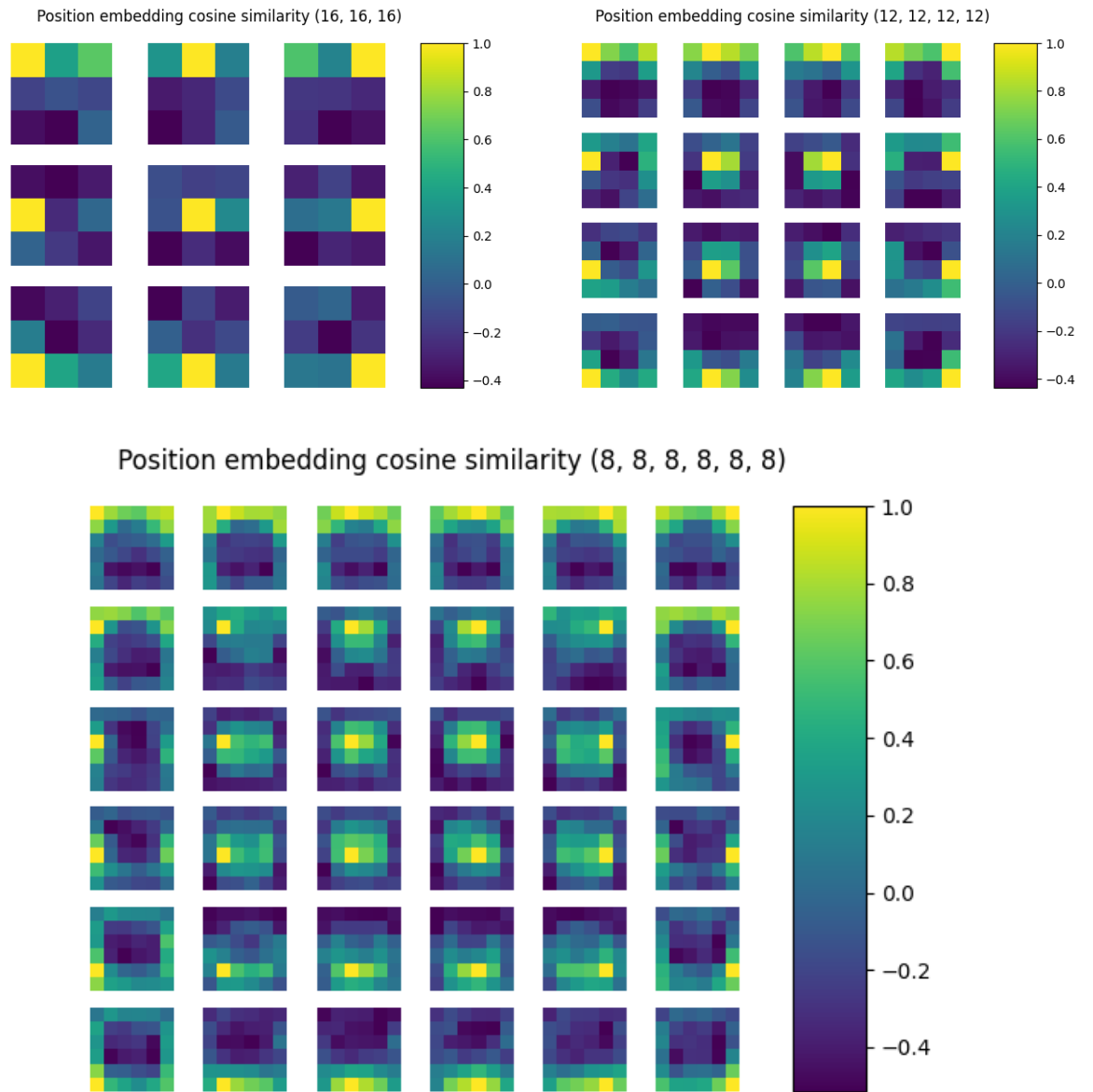


Figure 1: Unbiased patch sizes

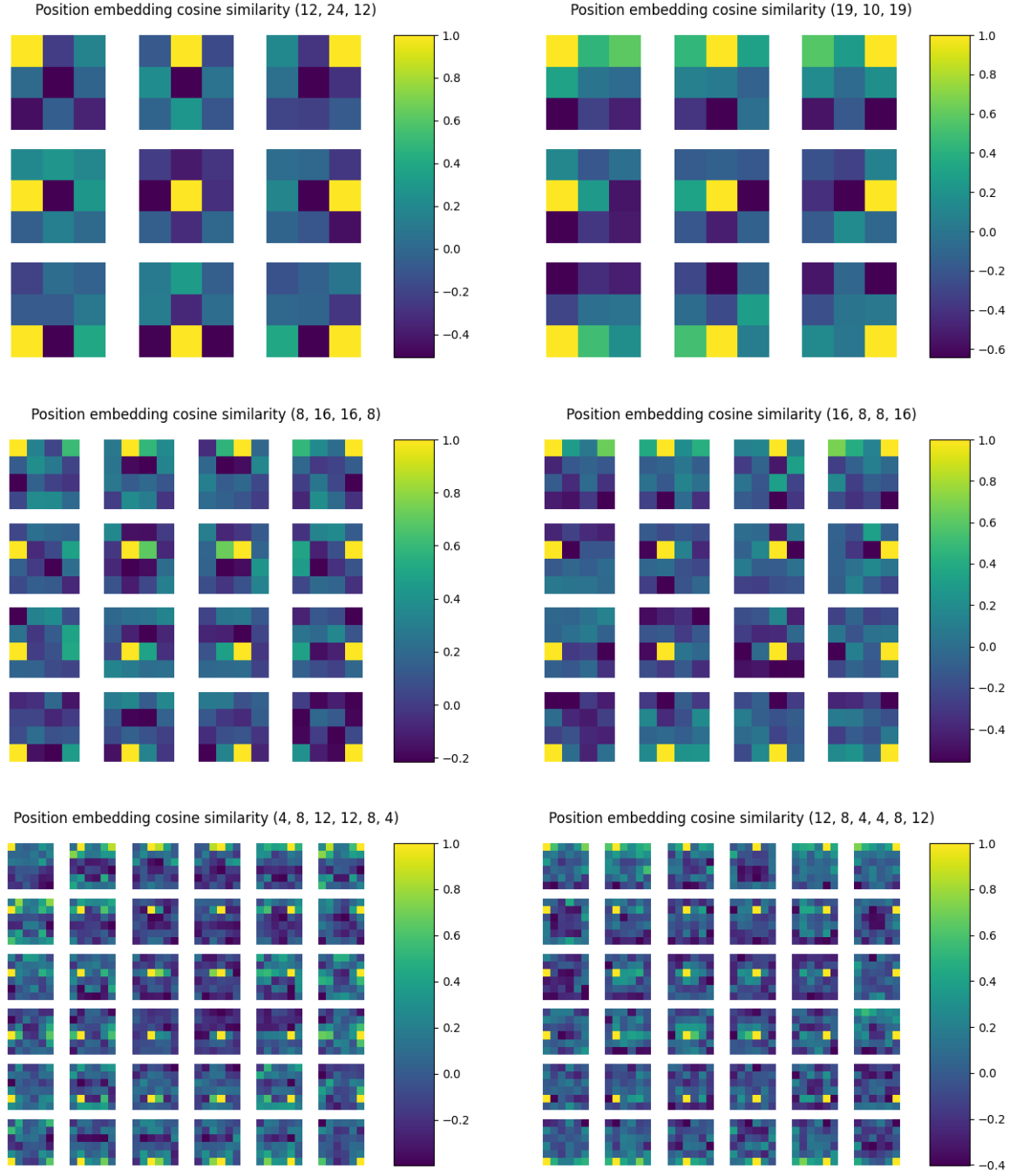


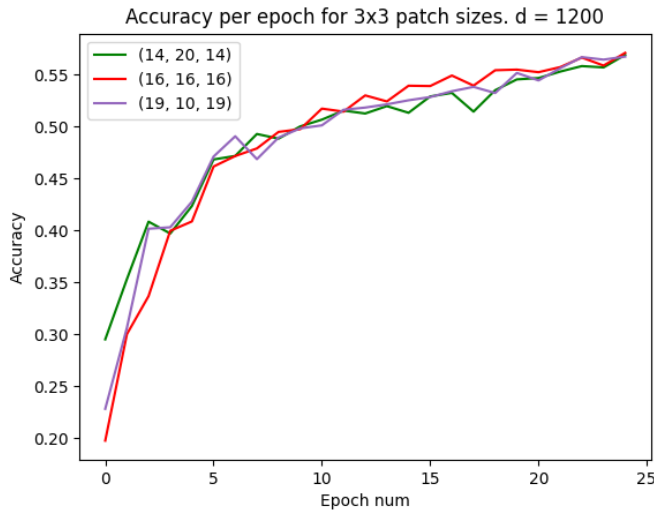
Figure 2: Biased patch sizes

As can be seen in the results, the models with the unbiased patch sizes consistently start with the lowest accuracies, as they should according to the hypothesis

that they lack the inductive biases the biased models possess, and end with the highest accuracies. As claimed in the paper “An image is worth 16x16 words”, models with inductive biases are usually faster learners, but their capacity to learn should be lower than unbiased models. While I believe the fact that the unbiased models consistently have the lowest accuracies at the start is consistent with that claim, I was surprised to see them consistently being the models with highest accuracies at the end of the training and seeing the biased models beginning to plateau, as I doubt the models were nearing their capacity to learn after merely 25 epochs. I believe one of the reasons for that phenomenon can be deduced by observing the cosine similarity of the learned position embeddings. The position embeddings learned by the unbiased models were able to achieve much higher similarity for neighboring patches than the ones learned by the biased models. Intuitively this is not very surprising, as patches of varying sizes, that are only partially adjacent and whose embeddings contain different amounts of the image data, will cause the learned position embeddings to have lower similarity. Since the unbiased models are able to learn meaningful position embeddings more quickly than the biased models, they end up achieving better results earlier than expected.

Another factor that might have an effect on the experiment results, is the fact that when embedding the patches to 256 dimensional vectors, the biased models lose more of the image information than the unbiased ones (since the large batches that contain most of the image information are embedded to the same dimension as the small patches). This led me to perform the second experiment, in which I trained 3 non-separate patch embeddings models with an embedding dimension $d = 1200$. Since the biggest patch size in any of the models was 1200 ($20 * 20 * 3$), none of the models lost any image data when embedding the patches.

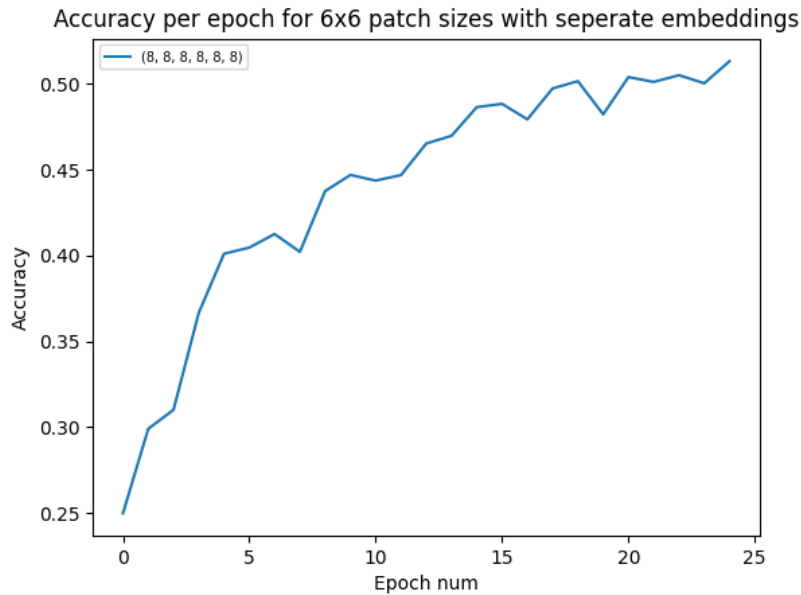
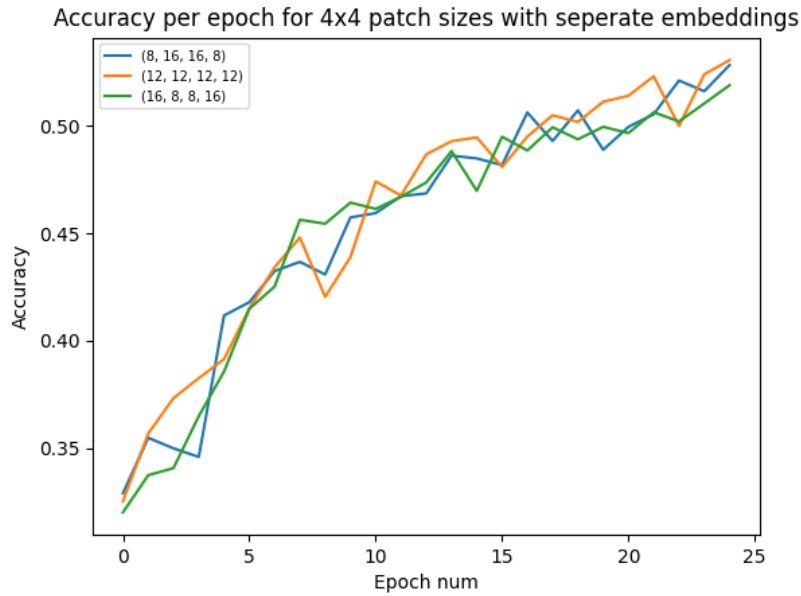
The accuracy results:



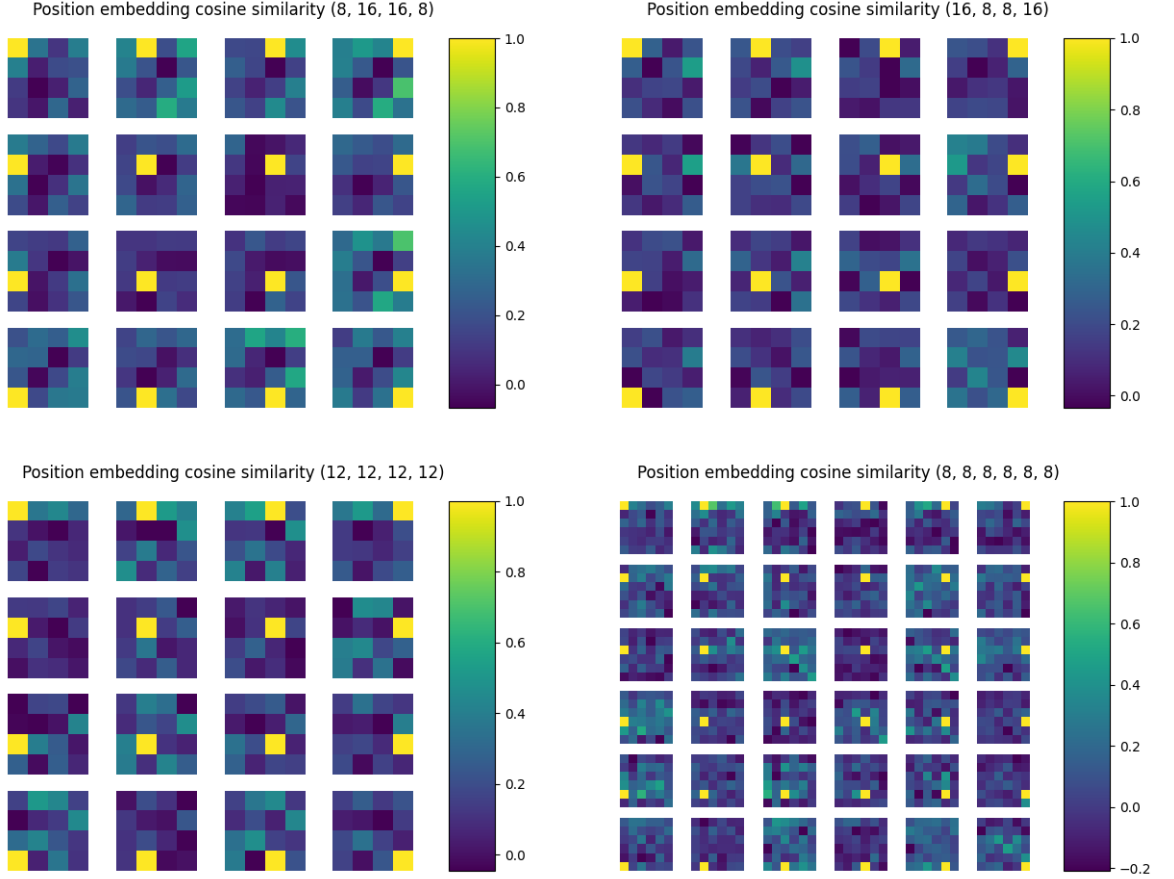
As shown in the figure, having none of the embeddings lose image data indeed reduced the gap between the biased and unbiased models past the first few epochs, but the unbiased model still seems a bit stronger overall in the later epochs.

The last experiment involved training a few separate patch embeddings models, and seeing the difference in results using different linear embedding layers for each patch makes. Embedding dimension was $d = 256$.

The accuracy results:



Position embeddings cosine similarities:



As we can see all the models' performances were noticeably worse. Embedding the patches using separate parameter caused the models to barely be able to learn any meaningful position embeddings within the 25 training epochs. Despite achieving worse results in the experiment, this method likely increases the models' capacity to learn by increasing the number of learned parameters and by possibly removing even more inductive bias from the model (not forcing all the patches to be embedded using the same linear projection).

5 Conclusion

While this experiment was conducted using limited computing power, and despite there being some limitations and unresolved problems regarding the method (especially the poor ability to learn meaningful position embeddings), the experiment was able to show the potential of biased patch division as a tool for injecting induc-

tive bias into image models and enabling faster learning (while trading-off learning capacity).

Further research into the method and ways to circumvent its shortcomings using more powerful hardware is required before harnessing it for deep learning purposes. Hopefully in the future it could be of use to researchers in the field.